INTER IIT TECH MEET 15.0

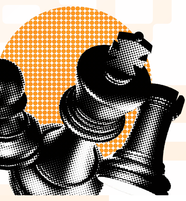
IIT Guwahati Tech Board

Inter IIT Bootcamp

#GoldforGuwahati

Phase 1

Software PS



- Team Size up to 1-3 Members
- Mentorship Session
- Submission Deadline 7th October

INTER IIT TECH MEET 15.0

IIT Guwahati Tech Board

CalcInk: On-Device Handwritten Math Calculator

Track: Software Development
Domain: Digital Ink, HCI & Edge ML
Mode: 100% Client-Side

1. Project Details

Modern digital paper platforms and note-taking applications (such as Apple iPadOS Math Notes and MyScript Nebo) are transforming passive sketch surfaces into active computational environments. Rather than manually copying handwritten formulas into separate calculator tools, users increasingly expect digital paper to interpret mathematical handwriting naturally and calculate results directly on the page.

In this challenge, teams must engineer CalcInk, a responsive web-based digital notebook that enables users to write mathematical equations by hand using a mouse, stylus, or touch. As the user sketches an expression ending with an equals sign (such as 18+4×3 =), the application continuously captures the strokes, identifies the sequence of handwritten characters, evaluates the arithmetic expression, and projects the calculated result directly onto the canvas inline next to the equation.

The application is intended to function as an interactive computational scratchpad. If a user erases a term or replaces a number, the system must dynamically re-evaluate the equation and update the displayed answer in real time, bridging freehand sketching with immediate, deterministic computation.

2. Required Features

• **Interactive Digital Ink Canvas:** A fluid, low-latency drawing canvas supporting mouse, stylus, and touch inputs with smooth curve rendering. The canvas must include standard utility controls (undo/redo history, stroke and pixel eraser, clear canvas, and stroke width adjustment) and must account for high-DPI/Retina display scaling (window.devicePixelRatio) to maintain crisp strokes across varied viewports.

• **Handwriting Recognition & Math Vocabulary:** A recognition pipeline capable of identifying handwritten expressions consisting of digits (0–9), standard arithmetic operators (+, −, ×, ÷), decimal points (.), and the terminal equals sign (=). Recognition must be powered by an existing open-source pre-trained model integrated directly into the client application.

• **Mathematical Evaluation Engine:** A deterministic arithmetic parser that evaluates recognized expressions following standard operator precedence (BODMAS/PEMDAS). The engine must support multi-digit integers, floating-point decimals, and negative numbers.

• **Dynamic Canvas Projection & Reactive Editing:** The evaluated answer must be rendered directly onto the canvas surface, positioned immediately adjacent to the terminal equals sign. If the user edits an existing equation (e.g., erasing a number and writing a replacement), the system must dynamically re-evaluate the expression and update the inline answer.

3. Limitations & Restrictions

• **100% On-Device Execution (Zero Cloud APIs):** All application workloads—stroke capture, image preprocessing, neural network inference, and mathematical evaluation—must run entirely client-side inside the user's browser. The use of cloud-hosted vision or math APIs (e.g., OpenAI Vision, Mathpix, Google Cloud Vision) or custom remote inference servers is strictly prohibited.

• **Complete Offline Operation:** Once the web application assets are loaded, the entire application lifecycle must function with zero network connectivity (verifiable in airplane mode).

• **60 FPS Frame Rate Integrity:** Heavy computational tasks such as image parsing and model inference must never block the primary browser thread. Drawing on the canvas must remain locked at a fluid 60 FPS without perceptible stutter or stylus lag while recognition is taking place.

• **Safe Execution & Fault Tolerance:** Direct use of un-sanitized JavaScript eval() is strictly forbidden. The system must handle edge cases gracefully, such as displaying a clean Undefined indicator for division by zero and handling malformed syntax without throwing unhandled runtime exceptions.

4. Important Notes

• **No Model Training from Scratch:** Teams are strictly not required or expected to train machine learning models from scratch. Participants are encouraged to source, integrate, and bundle existing open-source pre-trained models available online (e.g., from Hugging Face, GitHub, ONNX Model Zoo, or TensorFlow Hub).

• **Focus on Software Engineering:** The core evaluation centers on software craft, system integration, canvas performance, and user experience, rather than machine learning research or model development.

• **Runtime & Library Flexibility:** Teams have full autonomy to select any client-side web runtimes (such as ONNX Runtime Web, TensorFlow.js, Transformers.js, or WebAssembly) and frontend frameworks (React, Vue, Svelte, or vanilla web APIs) that fulfill the functional and performance constraints.

5. Submission Deliverables

• **Source Code Repository:** A clean GitHub/GitLab repository containing modular, documented source code and configuration files.

• **Project Documentation (README.md):** Clear quick-start instructions to install dependencies and run the application locally (e.g., npm install && npm run dev), alongside model attribution detailing the chosen pre-trained model's source link, license, and architecture.

• **Live Hosted Demonstration:** A publicly accessible deployment link (e.g., Vercel, Netlify, Cloudflare Pages, or GitHub Pages) for real-time evaluation.

6. Evaluation Rubric & Scoring Breakdown

Submissions will be evaluated out of a total of 100 Points based on technical execution, software architecture, engineering discipline, and overall polish.

Evaluation Pillar	Core Assessment Focus	Weightage
1. Feature Implementation & Test Suites	Completeness of canvas, recognition, math engine & automated test suites	25%
2. Idea & Architecture Design Document	System architecture, model selection rationale & design clarity	20%
3. Performance & Runtime Constraints	60 FPS frame rate, non-blocking inference & zero-cloud offline execution	20%
4. Teamwork & Engineering Discipline	Balanced Git history, modular code organization & clean documentation	15%
5. Creativity & User Experience	UI polish, micro-interactions & thoughtful bonus capabilities	20%
Total		100%

• Feature Implementation & Test Suites (25 Points):

– Complete delivery of all required canvas features (smooth drawing, responsive undo/redo stack, eraser, and display scaling).

– High recognition accuracy across all required digits (0–9), arithmetic operators, decimals, and equals delimiters.

– Correct arithmetic parsing following BODMAS/PEMDAS order of operations with support for multi-digit and decimal numbers.

– Presence of comprehensive automated test suites (e.g., unit tests for the arithmetic parser, edge-case validation for division by zero, and coordinate conversion testing).

• Idea & Architecture Design Document (20 Points):

– Quality of the architectural design document in the repository (README.md or dedicated architecture document).

– Clear technical justification for the chosen pre-trained model (reasoning regarding model size, inference speed, accuracy, and evaluated alternatives).

– Thoughtful pipeline design bridging raw canvas stroke coordinates with neural network input tensors.

• Performance, Responsiveness & Runtime Constraints (20 Points):

– Strict adherence to the 60 FPS frame budget: zero perceptible input lag, pen stutter, or dropped frames during active recognition.

– Robust non-blocking execution (effective decoupling of heavy computational inference from the main UI thread).

– Strict compliance with the 100% on-device mandate: verified full functionality in offline/airplane mode without remote API calls.

– Memory stability with zero memory leaks during prolonged drawing sessions.

• Teamwork & Engineering Discipline (15 Points):

– Evident division of labor and balanced Git contribution history across all participating team members.

– Healthy software engineering hygiene: meaningful commit messages, structured pull requests, and modular project organization.

– Effortless reproducibility: clear setup instructions that allow judges to build and run the project locally without environment debugging.

• Creativity & User Experience (20 Points):

– Visual elegance, typography, and intuitive digital paper aesthetic (simulating natural paper physics).

– Thoughtful micro-interactions, such as audio/haptic feedback, stroke animation, or confidence indicators.

– Creative extensions that elevate the product beyond the baseline (e.g., gesture-based scratch-to-erase, variable memory support such as x = 10, or interactive 2D function curve plotting).